

COMP0171 Bayesian Deep Learning — 2025 Past Paper

Model Answers

Contents

1	Question 1: True/False [14 marks total]	2
2	Question 2: Short Answer [26 marks total]	5
3	Question 3: Proposal Distributions [15 marks total]	9
4	Question 4: Bayesian Variational Auto-Encoder [25 marks total]	12
5	Question 5: Gradients, Autodiff, and Recursion [20 marks total]	16

1 Question 1: True/False [14 marks total]

Please identify if statements are either True or False. Please justify your answer if false. Correct “True” questions yield 2 points. Correct “False” questions yield one point for the answer and one point for the justification.

1(a) [2 marks]

(T/F) Stochastic gradient Langevin Dynamics is typically used in big data settings, despite being an inexact sampler, as it only uses a small subset of the data on every iteration.

True.

SGLD 将 Langevin dynamics 与 stochastic gradient 结合。标准 MCMC (如 Metropolis-Hastings) 每步需遍历全部数据来计算 acceptance ratio, 在大数据场景下成本极高。SGLD 的做法是:

1. 用 minibatch gradient 代替 full-batch gradient 来近似 $\nabla_{\theta} \log \pi(\theta)$;
2. 跳过 accept/reject step, 直接用 proposal 作为下一个 sample。

更新公式为:

$$\theta^{(t+1)} = \theta^{(t)} + \eta \nabla_{\theta} \log \pi(\theta^{(t)}) + \sqrt{2\eta} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

跳过 accept/reject 使其成为 **inexact sampler**, 但在 $\eta \rightarrow 0$ 极限下趋于精确的 Langevin diffusion。使用有限 η 和 minibatch 噪声是一种近似但高效的采样方法, 特别适合 big data settings。

1(b) [2 marks]

(T/F) Averages of samples from Markov chain Monte Carlo (MCMC) can be used to compute posterior expectations and estimate marginal likelihoods.

False.

MCMC 样本可以计算 posterior expectations, 这没问题: 若 $\theta^{(1)}, \dots, \theta^{(S)} \sim p(\theta|\mathcal{D})$, 则

$$\mathbb{E}_{p(\theta|\mathcal{D})}[f(\theta)] \approx \frac{1}{S} \sum_{s=1}^S f(\theta^{(s)}).$$

但 MCMC 不能直接估计 marginal likelihood $p(\mathcal{D})$ 。原因在于 MCMC 从 posterior $p(\theta|\mathcal{D}) \propto p(\mathcal{D}|\theta)p(\theta)$ 中采样时, normalizing constant $p(\mathcal{D})$ 在 acceptance ratio 中消去了:

$$\alpha = \min \left(1, \frac{\gamma(\theta') q(\theta|\theta')}{\gamma(\theta) q(\theta'|\theta)} \right), \quad \gamma(\theta) = p(\mathcal{D}|\theta)p(\theta),$$

MCMC 从不需要也不计算 $p(\mathcal{D})$ 。估计 marginal likelihood 需要额外方法, 如 importance sampling、thermodynamic integration 或 harmonic mean estimator 等。

1(c) [2 marks]

(T/F) A Gaussian process model with a linear kernel has the same predictive distribution as Bayesian linear regression, but will require less computation time and memory usage.

False.

前半句正确：GP with linear kernel $k(x, x') = x^\top x'$ 与 Bayesian linear regression 给出相同的 predictive distribution。这是 kernel trick 的经典结果。

但后半句错误。GP 的计算复杂度为 $O(N^3)$ (求逆 $N \times N$ 的 kernel matrix)，内存为 $O(N^2)$ 。Bayesian linear regression 的计算复杂度为 $O(D^3)$ (求逆 $D \times D$ 的矩阵)，内存为 $O(D^2)$ ，其中 D 是输入维度。对于 linear kernel, feature space 就是原始输入空间。当 $N > D$ 时 (通常如此)，**Bayesian linear regression** 更快更省内存，而非 GP。

1(d) [2 marks]

(T/F) Suppose you are training a machine learning model to identify the specific breed of a dog from a photo. The model mostly works well, but has low-confidence predictions on very rare breeds which only are present in a few training images. This low confidence is an example of epistemic uncertainty, which could be captured and quantified by using a Bayesian neural network.

True.

低置信度源于训练数据不足 (rare breeds 样本极少)，这正是 **epistemic uncertainty** (认知不确定性) 的定义：因对模型/参数的知识不足而产生的不确定性，可通过收集更多数据减少。

BNN 通过维护参数的 posterior distribution $p(\theta|\mathcal{D})$ 来捕捉 epistemic uncertainty。对数据稀少的类别，posterior 较宽 (高不确定性)，在 predictive distribution 中反映为：

$$p(y|x, \mathcal{D}) = \int p(y|x, \theta) p(\theta|\mathcal{D}) d\theta.$$

当 $p(\theta|\mathcal{D})$ 方差大时，不同 θ 给出不同预测 $\Rightarrow$ predictive distribution 分散 (低置信度)。

这不是 aleatoric uncertainty (固有数据噪声)，因为更多 rare breed 图片会提高置信度。

1(e) [2 marks]

(T/F) Forward-mode automatic differentiation is preferable to reverse-mode automatic differentiation for models with many outputs and few inputs.

True.

Forward mode: 一次 pass 计算对一个输入变量的导数，总成本 $\propto D_{in}$ 。

Reverse mode: 一次 pass 计算一个输出对所有参数的梯度，总成本 $\propto D_{out}$ 。

当 $D_{out} \gg D_{in}$ (many outputs, few inputs) 时，forward mode 只需少量 pass (每个输入一次)，远优于 reverse mode (需 D_{out} 次 backward pass)。

典型例子：计算 $f: \mathbb{R} \rightarrow \mathbb{R}^M$ 的 Jacobian — forward mode 一次 pass 即可，reverse mode 需要 M 次。

1(f) [2 marks]

(T/F) An advantage of using a convolutional layer for image processing is that the learned features are invariant to small translations of the inputs.

False.

卷积层的特征是 **equivariant** (等变) 而非 **invariant** (不变)。

Equivariance: 输入平移 $\Rightarrow$ 输出同样平移。**Invariance**: 输入平移 $\Rightarrow$ 输出不变。

形式化证明: 设 shifted input $x'_{d+1} = x_d$, 则

$$h'_{d+1} = \sigma(w_1 x'_d + w_2 x'_{d+1} + w_3 x'_{d+2}) = \sigma(w_1 x_{d-1} + w_2 x_d + w_3 x_{d+1}) = h_d.$$

输出也平移了一步 — 这是 **equivariance**, 不是 **invariance**。要获得 **translation invariance** 需要额外的 **pooling** 操作 (如 **global average pooling**)。

1(g) [2 marks]

(T/F) You are interested in training a spam filter, to identify and filter undesirable emails. You have a dataset of emails, but require a human labeller to come in and manually tell you what emails are spam and which are not. Active learning would not be possible in this setting, since the emails are discrete data (not real-valued inputs).

False.

Active learning 完全可以应用于离散数据。其核心思想是: 基于 **model uncertainty** 智能选择哪些样本让人类标注, 以最大化模型性能提升。选择标准 (如 **entropy**、**mutual information**) 与输入是否连续无关。

在 **spam filter** 场景中: 有大量未标注 emails (**pool of unlabelled data**), 当前模型对某些 email 分类不确定, **active learning** 选择这些 **high-uncertainty** 的 email 让标注员优先标注。

Active learning 的适用条件: (1) 有 **unlabelled data pool**, (2) **labelling** 有成本, (3) 模型能量化 **uncertainty**。这些条件都不要求输入是 **real-valued** 的。文本数据 (经 **embedding** 后) 同样可以计算 **epistemic uncertainty** 并用于 **active learning**。

2 Question 2: Short Answer [26 marks total]

Please give brief answers to the following questions.

2(a) [3 marks]

U-nets (for example, for image segmentation) and autoencoders (for example, for unsupervised representation learning) both take high dimensional data, transform it to a lower dimensional space, and then transform it back. A major difference is the inclusion of skip connections in the U-Net. What are these connections for? Why don't autoencoders include them? Should they?

Skip connections 的作用：将 encoder 各层的 high-resolution spatial features 直接传递给 decoder 中对应分辨率的层。对 image segmentation 等像素级预测任务，精细空间细节（边缘、纹理、位置信息）至关重要，而这些信息经过 bottleneck 时不可避免地会丢失。Skip connections 允许 decoder 直接访问这些细节，恢复精确的空间定位。

Autoencoder 为什么不加：Autoencoder 的核心目标是通过 bottleneck 强制学习 compressed representation（压缩表示）。如果加了 skip connections，信息可以直接绕过 bottleneck 从 encoder 流向 decoder，网络可以“偷懒”直接复制而不学习有意义的 representation — 完全违背了 autoencoder 的设计目的。

应该加吗？不应该。Bottleneck 是 autoencoder 的核心 feature 而非缺陷。移除它就失去了学习 compressed representation 的动力。

2(b)(i) [3 marks]

Recurrent neural networks (RNNs): What is the difference between a standard RNN and a bidirectional RNN?

Standard RNN 只沿一个方向（从左到右）处理序列。时刻 t 的 hidden state 仅依赖当前输入和之前的 hidden state:

$$h_t = g(W_h h_{t-1} + W_x x_t + b).$$

因此 h_t 只包含 $x_1, \dots, x_t$ 的信息 (past context only)。

Bidirectional RNN 使用两个独立的 RNN：一个 forward RNN (从左到右)，一个 backward RNN (从右到左)。时刻 t 的表示是两者的拼接：

$$h_t^F = g(W_h^F h_{t-1}^F + W_x^F x_t + b^F), \quad h_t^B = g(W_h^B h_{t+1}^B + W_x^B x_t + b^B),$$

$$h_t = [h_t^F; h_t^B].$$

因此 h_t 编码了整个序列的上下文信息 (past + future context)。

2(b)(ii) [2 marks]

When would a bidirectional RNN be preferred?

Bidirectional RNN 适合整个输入序列已完全可用的任务，且需要每个位置都融合双向上下文。典型例子：sequence classification (如 sentiment analysis)，sequence labelling / tagging (如

NER、POS tagging), machine translation 中的 encoder 部分。核心条件是: 推理时可以看到完整序列, 而非逐 token 生成。

2(b)(iii) [2 marks]

When would a standard RNN be preferred?

Standard (unidirectional) RNN 适合 **autoregressive generation** (自回归生成) 任务。典型例子: language modelling (预测 $p(x_{t+1}|x_1, \dots, x_t)$)、text generation、online / streaming 处理 (数据逐步到达, 无法等到序列结束)。Bidirectional RNN 无法用于生成任务, 因为生成第 t 个 token 时尚不知道未来 token $x_{t+1}, x_{t+2}, \dots$

2(c)(i) [3 marks]

Consider a regression problem where we have a dataset $\mathcal{D} = \{(x_1, y_1), \dots, (x_N, y_N)\}$, where $x_i \in \mathbb{R}^D$ and $y_i \in \mathbb{R}$. We would like to model the data using a likelihood $p(y_i|x_i, \theta)$ and a prior $p(\theta)$.

What objective function would you optimise to perform MAP (maximum a posteriori) estimation of θ ? Write an equation.

MAP estimation 最大化 posterior:

$$\theta_{\text{MAP}} = \arg \max_{\theta} p(\theta|\mathcal{D}) = \arg \max_{\theta} p(\mathcal{D}|\theta) p(\theta).$$

取对数后等价于最大化 ($\log p(\mathcal{D})$ 是常数, 不影响优化):

$$\theta_{\text{MAP}} = \arg \max_{\theta} \left\{ \sum_{i=1}^N \log p(y_i|x_i, \theta) + \log p(\theta) \right\}.$$

第一项是 log-likelihood (数据拟合项), 第二项是 log-prior (正则化项)。

2(c)(ii) [3 marks]

Suppose the likelihood is a Gaussian centered at the output of a neural network and there is a Gaussian prior on θ , i.e.

$$p(\theta) = \mathcal{N}(\theta|0, \alpha^{-1}I), \quad p(y|x_i, \theta) = \mathcal{N}(y|f(x_i, \theta), \beta^{-1}),$$

where α, β are fixed hyperparameters. Show that MAP estimation of θ is equivalent to minimising a squared error loss with a quadratic regulariser.

展开 log-posterior (忽略与 θ 无关的常数项):

$$\begin{aligned} \log p(\theta|\mathcal{D}) &= \sum_{i=1}^N \log p(y_i|x_i, \theta) + \log p(\theta) + \text{const} \\ &= \sum_{i=1}^N \left[-\frac{\beta}{2} (y_i - f(x_i, \theta))^2 \right] + \left[-\frac{\alpha}{2} \|\theta\|^2 \right] + \text{const}. \end{aligned}$$

最大化上式等价于最小化（取负号、除以 $\beta/2$ ）：

$$\mathcal{L}(\theta) = \sum_{i=1}^N (y_i - f(x_i, \theta))^2 + \frac{\alpha}{\beta} \|\theta\|^2.$$

第一项是 **squared error loss**，第二项是 **quadratic regulariser**（即 L_2 regularisation / weight decay），正则化强度为 $\lambda = \alpha/\beta$ （prior precision 与 likelihood precision 之比）。

2(c)(iii) [5 marks]

Repeatedly during the lectures, I made a statement “MAP estimation is not Bayesian”, despite the fact that it has a prior. Why would I say such a thing — what is not “Bayesian” about MAP estimation? What differentiates MAP estimation from a Bayesian approach?

虽然 MAP estimation 使用了 prior，但它不是 **Bayesian** 方法，原因如下：

1. 点估计 vs 分布：MAP 只给出一个 point estimate θ_{MAP} （posterior 的 mode），而 Bayesian approach 维护完整的 posterior distribution $p(\theta|\mathcal{D})$ 。信息被极大地压缩了。

2. 不量化 **epistemic uncertainty**：MAP 预测 $\hat{y} = f(x, \theta_{\text{MAP}})$ 是确定性的，无法表达模型对参数有多不确定。Bayesian 方法通过 posterior predictive distribution 自然地量化不确定性：

$$p(y|x, \mathcal{D}) = \int p(y|x, \theta) p(\theta|\mathcal{D}) d\theta.$$

3. 不 **marginalise**：Bayesian inference 的核心操作是 marginalisation（对 nuisance parameters 积分），而 MAP 只做 optimisation。这导致 MAP 无法做 model comparison（marginal likelihood 需要积分）。

4. 过拟合风险：MAP 可能找到 posterior 中一个非常尖锐的 peak，但该 peak 可能代表一个非典型区域（overfitting）。Bayesian 方法通过对 posterior 积分，自然实现 “Bayesian Occam’s Razor”，偏好占据 posterior 质量较大的简单解。

总结：Bayesian 的核心是 **posterior distribution + marginalisation**，而 MAP 只借用了 prior 做 regularised optimisation，本质仍是一个优化问题。

2(d)(i) [2 marks]

In the same regression setting as part (c), suppose you have trained the deep learning model K times, starting from different random initialisations, performing MAP estimation of the parameters using Adam as an optimiser. This yields K sets of parameters $\theta_1, \dots, \theta_K$.

Are these values $\theta_1, \dots, \theta_K$ the same or different? Why?

不同。原因有二：

(1) **Non-convex loss surface**：神经网络的 loss landscape 高度非凸，存在大量 local minima 和 saddle points。不同 random initialisation 导致优化器沿不同路径收敛到不同的 local minimum。

(2) **Stochastic optimisation**：Adam 使用 stochastic minibatch gradients，即使 initialisation 相同，不同 random seed 下 minibatch 采样的随机性也会导致不同的优化轨迹和最终参数。

这正是 deep ensembles 方法的基础：不同 initialisation 产生的多样化模型可以组合起来近似 posterior uncertainty。

2(d)(ii) [3 marks]

For each individual model parameter θ_k , suppose you can make predictions $\hat{y}_k = f(x, \theta_k)$, for a test input x . How would you evaluate $\Pr(y > 0|x)$ using this model?

每个模型的 predictive distribution 为 $p(y|x, \theta_k) = \mathcal{N}(y | f(x, \theta_k), \beta^{-1})$ 。
用 deep ensemble 近似 posterior predictive distribution:

$$p(y|x, \mathcal{D}) \approx \frac{1}{K} \sum_{k=1}^K \mathcal{N}(y | f(x, \theta_k), \beta^{-1}).$$

因此:

$$\Pr(y > 0|x) \approx \frac{1}{K} \sum_{k=1}^K \Pr(y > 0|x, \theta_k) = \frac{1}{K} \sum_{k=1}^K \Phi\left(\frac{f(x, \theta_k)}{\beta^{-1/2}}\right),$$

其中 $\Phi(\cdot)$ 是标准正态分布的 CDF。即对每个 ensemble member 分别计算 $\Pr(y > 0)$ (利用 Gaussian CDF)，然后取平均。

也可以通过 Monte Carlo 实现：对每个 θ_k 采样 $y_k \sim \mathcal{N}(f(x, \theta_k), \beta^{-1})$ ，统计 $y_k > 0$ 的比例。

3 Question 3: Proposal Distributions [15 marks total]

Importance sampling and MCMC are both Bayesian inference algorithms that use proposal distributions. Suppose we are trying to compute expectations under a probability distribution $\pi(x) = \frac{1}{Z}\gamma(x)$, with $\int \gamma(x) dx = Z$ and $x \in \mathbb{R}$. Assume we can evaluate $\gamma(x)$ for any particular value of x , but Z is unknown.

3(a)(i) [2 marks]

Suppose we would like to compute expectations using an importance sampling proposal $q(x) = \mathcal{N}(x|\mu, \sigma^2)$; that is, we will propose from a Gaussian distribution with mean μ and variance σ^2 . At a high level, how would you characterise a good proposal?

一个好的 importance sampling proposal $q(x)$ 应满足：

(1) 覆盖 **target** 的 **support**：凡 $\pi(x) \neq 0$ 处都需要 $q(x) > 0$ ，否则这些区域的贡献永远被遗漏。

(2) 尾部至少和 **target** 一样重 (ideally heavier tails than π)：确保 importance weights $w(x) = \gamma(x)/q(x)$ 不会出现极端大值，避免 variance 爆炸。

(3) 形状尽量接近 $\pi(x)$ ：理想情况下 $q(x) \propto \pi(x)$ 时 importance weights 恒为常数，方差为零。

3(a)(ii) [2 marks]

What could be a criteria for selecting μ and σ ?

μ ：设置为 $\gamma(x)$ 的 mode，即 $\mu = \arg \max_x \gamma(x)$ 。确保 proposal 中心与 target 高概率区域对齐。可通过优化 $\gamma(x)$ 或 Laplace approximation 确定。

σ ：应足够大以覆盖 $\pi(x)$ 的质量分布区域。一种方法是使用 Laplace approximation 中 Hessian 的逆来匹配曲率： $\sigma^2 \approx [-\nabla^2 \log \gamma(x)|_{x=\mu}]^{-1}$ ，并适当加宽以确保尾部覆盖。

也可以通过最小化 importance weight 的方差 $\text{Var}_q[w(x)]$ 或 $\text{Var}_q[w(x)f(x)]$ 来选择 μ, σ 。

3(b)(i) [2 marks]

Suppose we would like to draw posterior samples using a random-walk Metropolis-Hastings algorithm, where we propose from a Gaussian distribution $q(x'|x)$ where x is the current state and x' is a proposed next state.

What would the acceptance ratio be for this Metropolis-Hastings algorithm?

一般 Metropolis-Hastings acceptance ratio 为：

$$\alpha(x', x) = \min\left(1, \frac{\pi(x') q(x|x')}{\pi(x) q(x'|x)}\right).$$

对于 Gaussian random walk proposal $q(x'|x) = \mathcal{N}(x'|x, \sigma^2)$ ，由 Gaussian 的对称性 $q(x'|x) = q(x|x')$ ，proposal 项消去：

$$\alpha(x', x) = \min\left(1, \frac{\pi(x')}{\pi(x)}\right) = \min\left(1, \frac{\gamma(x')}{\gamma(x)}\right).$$

注意 $\pi = \gamma/Z$, normalizing constant Z 在比值中消去, 所以只需 unnormalised density γ 。

3(b)(ii) [2 marks]

Based off what we discuss in class and implemented in coursework, at a high level how would you characterise a good proposal $q(x'|x)$?

好的 random-walk proposal 的关键是 **step size** σ 的选择, 需要在“探索范围”和“接受率”之间平衡:

σ 太小: 几乎所有 proposal 被接受 (acceptance rate ≈ 1), 但每步移动太小, chain 需要很长时间才能充分探索 target distribution — **mixing** 太慢。

σ 太大: proposal 经常跳到低概率区域, 大部分被拒绝 (acceptance rate ≈ 0), chain 长时间停留在同一位置 — 同样 **mixing** 太慢。

理想的 σ 应使 acceptance rate 在适中范围内 (经验法则: 一维约 44%, 高维约 23%)。

3(c)(i) [2 marks]

Now consider a new Metropolis-Hastings algorithm, that we will call Independent Metropolis-Hastings. In this algorithm, instead of proposing from a Gaussian $q(x'|x)$, we propose from a Gaussian distribution $q(x') = \mathcal{N}(x'|\mu, \sigma^2)$ which does not depend on the current state x .

What would the acceptance ratio be for this Metropolis-Hastings algorithm?

Independent MH 的 proposal $q(x') = \mathcal{N}(x'|\mu, \sigma^2)$ 不依赖当前状态 x 。代入一般 MH acceptance ratio (注意 $q(x'|x) = q(x')$, $q(x|x') = q(x)$):

$$\alpha(x', x) = \min\left(1, \frac{\pi(x') q(x)}{\pi(x) q(x')}\right).$$

用 unnormalised density $\gamma = Z\pi$ 改写:

$$\alpha(x', x) = \min\left(1, \frac{\gamma(x')/q(x')}{\gamma(x)/q(x)}\right) = \min\left(1, \frac{w(x')}{w(x)}\right),$$

其中 $w(x) = \gamma(x)/q(x)$ 就是 importance weight。Acceptance ratio 等于 proposed 和 current state 的 importance weight 之比。

3(c)(ii) [2 marks]

What choice of $q(x)$ would maximise the acceptance ratio (bringing it close to 1)? Would this be a good choice?

若令 $q(x) = \pi(x)$ (proposal 就是 target), 则对任意 x, x' :

$$\frac{\gamma(x')/q(x')}{\gamma(x)/q(x)} = \frac{\gamma(x')/\pi(x')}{\gamma(x)/\pi(x)} = \frac{Z}{Z} = 1.$$

因此 $\alpha = 1$, 所有 proposal 都被接受。

但这不是一个可行的选择: 如果能直接从 $\pi(x)$ 中采样, 就根本不需要 MCMC 了! 这是一个 **circular argument** (循环论证) — MCMC 的目的恰恰是从我们无法直接采样的分布中获

取样本。

3(d) [3 marks]

Give one example of a setting where you would have a strong preference as to whether to use random-walk or independent proposal Metropolis-Hastings, and explain why.

场景：高维 **posterior inference**（如 BNN 权重空间，维度 $d \gg 1$ ）— 应偏好 **random-walk MH**。

原因：在高维空间中，independent MH 的固定 proposal $q(x') = \mathcal{N}(\mu, \sigma^2 I)$ 很难覆盖 target distribution 的高概率区域。Importance weights $w(x) = \gamma(x)/q(x)$ 的方差随维度指数增长 (curse of dimensionality)，导致绝大部分 proposal 的 weight 极小，acceptance rate 趋近于 0。

相反，random-walk MH 通过局部移动来探索分布，每一步只在当前位置附近 propose，避免了高维空间中“大海捞针”的问题。虽然探索速度慢 (diffusive behavior)，但至少能稳定工作。

反例：如果有一个很好的全局近似（如通过 Laplace approximation 或 variational inference 已经得到接近 π 的分布），则 **independent MH** 更好 — 它可以做大跳跃，不受 local exploration 的限制，mixing 更快。

4 Question 4: Bayesian Variational Auto-Encoder [25 marks total]

In class, we discussed variational autoencoders. In contrast to “standard” auto-encoders, they represent a Bayesian treatment of the latent variables in the learned representation. However, the neural network defining the generative model (i.e., the decoder) was not treated “Bayesian” at all! Let’s fix that.

4(a) [4 marks]

Assume we have data $x_i, i = 1, \dots, N$, an encoder $q(z|x, \phi)$, and a decoder $p(x|z, \theta)$, where θ are decoder neural network parameters and ϕ are encoder neural network parameters. Assuming some prior $p(z)$ on the latent space, derive the standard ELBO for a variational autoencoder from the lectures and coursework — that is, the lower bound on the marginal likelihood $\sum_{i=1}^N \log p(x_i|\theta)$, where $p(x_i|\theta) = \int p(x_i|z_i, \theta) p(z_i) dz_i$.

对单个数据点 x_i , 引入 variational posterior $q_\phi(z|x_i)$:

$$\begin{aligned} \log p(x_i|\theta) &= \log \int p(x_i|z, \theta) p(z) dz \\ &= \log \int \frac{p(x_i|z, \theta) p(z)}{q_\phi(z|x_i)} q_\phi(z|x_i) dz \\ &\geq \int q_\phi(z|x_i) \log \frac{p(x_i|z, \theta) p(z)}{q_\phi(z|x_i)} dz \quad (\text{Jensen's inequality, log is concave}) \\ &= \mathbb{E}_{q_\phi(z|x_i)} [\log p(x_i|z, \theta)] + \mathbb{E}_{q_\phi(z|x_i)} \left[\log \frac{p(z)}{q_\phi(z|x_i)} \right] \\ &= \mathbb{E}_{q_\phi(z|x_i)} [\log p(x_i|z, \theta)] - \text{KL}(q_\phi(z|x_i) \| p(z)). \end{aligned}$$

对所有数据点求和得到 VAE ELBO:

$$\sum_{i=1}^N \log p(x_i|\theta) \geq \sum_{i=1}^N \left\{ \underbrace{\mathbb{E}_{q_\phi(z|x_i)} [\log p(x_i|z, \theta)]}_{\text{reconstruction term}} - \underbrace{\text{KL}(q_\phi(z|x_i) \| p(z))}_{\text{KL regulariser}} \right\} \equiv \mathcal{L}(\phi, \theta).$$

第一项鼓励 decoder 在 encoder 给出的 latent code 下重建数据, 第二项鼓励 approximate posterior 接近 prior $p(z)$, 防止 latent space 退化。

4(b) [2 marks]

Now, suppose some prior $p(\theta)$ over the decoder parameters. What is the joint distribution defined by this model?

引入 decoder 参数的 prior $p(\theta)$ 后, 完整的 joint distribution 为:

$$p(\theta, z_{1:N}, x_{1:N}) = p(\theta) \prod_{i=1}^N p(z_i) p(x_i|z_i, \theta).$$

即：先从 prior 生成 decoder 参数 $\theta \sim p(\theta)$ ，然后对每个数据点独立地生成 latent variable $z_i \sim p(z_i)$ ，再通过 decoder 生成观测 $x_i \sim p(x_i|z_i, \theta)$ 。数据点之间条件独立 (given θ)。

4(c) [6 marks]

Suppose we would like to learn an approximate posterior $q_\lambda(\theta|x_1, \dots, x_N) \approx p(\theta|x_1, \dots, x_N)$ over the decoder network parameters, where λ represents the new variational parameters. Derive out an appropriate ELBO that could be optimised to learn this “fully Bayesian” VAE decoder.

目标：推导 $\log p(x_{1:N})$ 的下界，其中

$$p(x_{1:N}) = \int p(\theta) \prod_{i=1}^N \int p(z_i) p(x_i|z_i, \theta) dz_i d\theta.$$

引入 joint variational distribution, 假设 factorised 形式:

$$q(\theta, z_{1:N}|x_{1:N}) = q_\lambda(\theta) \prod_{i=1}^N q_\phi(z_i|x_i).$$

由 $\text{KL}(q||p_{\text{posterior}}) \geq 0$, 标准 ELBO 推导给出:

$$\begin{aligned} \log p(x_{1:N}) &\geq \mathbb{E}_{q(\theta, z_{1:N})} \left[\log \frac{p(\theta, z_{1:N}, x_{1:N})}{q(\theta, z_{1:N}|x_{1:N})} \right] \\ &= \mathbb{E}_{q(\theta, z_{1:N})} \left[\log \frac{p(\theta) \prod_i p(z_i) p(x_i|z_i, \theta)}{q_\lambda(\theta) \prod_i q_\phi(z_i|x_i)} \right] \\ &= \mathbb{E}_{q_\lambda(\theta)} \left[\log \frac{p(\theta)}{q_\lambda(\theta)} \right] + \sum_{i=1}^N \mathbb{E}_{q_\phi(z_i|x_i)} \left[\log \frac{p(z_i)}{q_\phi(z_i|x_i)} \right] + \sum_{i=1}^N \mathbb{E}_{q_\lambda(\theta)} \left[\mathbb{E}_{q_\phi(z_i|x_i)} [\log p(x_i|z_i, \theta)] \right]. \end{aligned}$$

整理后得到 Fully Bayesian VAE ELBO:

$$\mathcal{L}(\lambda, \phi) = \underbrace{\mathbb{E}_{q_\lambda(\theta)} \left[\sum_{i=1}^N \mathbb{E}_{q_\phi(z_i|x_i)} [\log p(x_i|z_i, \theta)] \right]}_{(1) \text{ expected reconstruction}} - \underbrace{\sum_{i=1}^N \text{KL}(q_\phi(z_i|x_i)||p(z_i))}_{(2) \text{ latent KL}} - \underbrace{\text{KL}(q_\lambda(\theta)||p(\theta))}_{(3) \text{ weight KL (new)}}.$$

三个部分的含义:

(1) **Expected reconstruction**: 在 $q_\lambda(\theta)$ 和 $q_\phi(z_i|x_i)$ 下的 expected log-likelihood — 注意与标准 VAE 相比, 现在 θ 也是随机的, 需要对其取期望。

(2) **Latent KL**: 与标准 VAE 完全相同, 正则化每个 z_i 趋向 prior $p(z_i)$ 。

(3) **Weight KL**: $\text{KL}(q_\lambda(\theta)||p(\theta))$ 是新增项, 正则化 decoder 权重的 variational posterior 趋向 prior, 实现 Bayesian weight regularisation。

4(d) [3 marks]

Write down an appropriate mini-batch estimator of the gradient you would use to train this model, using a subset of samples $\{x_1, \dots, x_M\}$.

对 minibatch $\{x_1, \dots, x_M\} \subset \{x_1, \dots, x_N\}$, data-dependent 项 (reconstruction 和 latent KL) 按 N/M 放大以无偏估计全数据的贡献, 而 weight KL 项只出现一次, 不需要放大:

$$\nabla_{(\lambda, \phi)} \hat{\mathcal{L}}_M = \nabla_{(\lambda, \phi)} \left\{ \frac{N}{M} \sum_{j=1}^M \left[\mathbb{E}_{q_\lambda(\theta)} \left[\mathbb{E}_{q_\phi(z_j|x_j)} [\log p(x_j|z_j, \theta)] \right] - \text{KL}(q_\phi(z_j|x_j) \| p(z_j)) \right] - \text{KL}(q_\lambda(\theta) \| p(\theta)) \right\}.$$

实际计算时用 **reparameterisation trick** 来估计期望:

- 对 θ : 采样 $\eta \sim p(\eta)$, 令 $\theta = h(\lambda, \eta)$ (如 $\theta = \mu_\lambda + \sigma_\lambda \odot \eta$, $\eta \sim \mathcal{N}(0, I)$);
- 对 z_j : 采样 $\epsilon_j \sim \mathcal{N}(0, I)$, 令 $z_j = g(\phi, x_j, \epsilon_j)$.

梯度通过 deterministic function 反传, 使得整个 ELBO 对 (λ, ϕ) 可微。

4(e) [3 marks]

Would you advocate this “Bayesian” variational auto-encoder approach? Specifically, what would you consider the main benefit (if any) relative to traditional training?

主要好处:

(1) 更好的 **calibration** 和抗 **overfitting**: $\text{KL}(q_\lambda(\theta) \| p(\theta))$ 项提供原则性的 weight regularisation (类似但优于手动 weight decay), 在小数据集上尤其有价值, 能有效防止 decoder 记忆训练数据。

(2) **Predictive uncertainty**: 生成新样本时可以对 θ 做 marginalisation, 生成分布自动反映 decoder 参数的不确定性。

(3) **Model selection**: ELBO 本身就是 marginal likelihood 的下界, 可以直接用于比较不同架构和超参数。

潜在顾虑: 训练成本更高 (需维护 $q_\lambda(\theta)$ 的参数, 每步需采样 θ); 如果数据量足够大, Bayesian 带来的收益可能有限 (posterior 会 concentrate 到一个 point)。因此这种方法在小/中等数据集上最有价值。

4(f) [4 marks]

Should we also do inference over the encoder parameters? If so, how? If not, why not?

不需要, 理由如下:

1. **Encoder** 不属于 **generative model**: encoder $q_\phi(z|x)$ 是 **variational approximation** (inference model), 不是 generative model $p(\theta, z, x)$ 的一部分。参数 ϕ 从不出现在 joint distribution $p(\theta) \prod_i p(z_i) p(x_i|z_i, \theta)$ 中, 因此从概率模型角度看, 对 ϕ 做 posterior inference 没有意义。

2. ϕ 是 **variational parameters**: encoder 参数 ϕ 与 λ ($q_\lambda(\theta)$ 的参数) 角色完全相同 — 它们都是 variational parameters, 目标是最大化 ELBO。对 variational parameters 本身再做 Bayesian inference 会导致 “inference about inference” 的无限回归。

3. 不影响生成质量: 训练完成后, 生成新样本只需 decoder $p(x|z, \theta)$ 和 prior $p(z)$, 完全不需要 encoder。Encoder 参数的不确定性不影响最终生成模型的质量。

如果执意要对 ϕ 做 inference, 可以将其视为另一层 variational parameter 并引入 $q(\phi)$, 但这会大大增加计算成本而没有明显好处。

4(g) [3 marks]

Do we have to do this all jointly with variational inference? Would some other posterior approximation for θ be possible or appropriate?

不必一定用 variational inference。其他可行的 posterior approximation 方法包括：

1. MCMC / SGLD: 对 decoder 参数 θ 使用 MCMC 采样 (如 stochastic gradient Langevin dynamics), 同时对 z_i 仍使用 amortised variational inference (encoder)。这是一种混合方法: MCMC 提供渐近精确的 posterior samples for θ , 而 encoder 提供高效的 approximate posterior for z 。

2. Deep ensembles: 训练 K 个不同 initialisation 的 VAE, 每个得到一组 θ_k 。用这些 $\{\theta_k\}_{k=1}^K$ 近似 posterior over decoder parameters。简单易实现, 无需修改训练流程。

3. Laplace approximation: 先做标准 MAP training 得到 θ_{MAP} , 然后在该点计算 Hessian, 用 Gaussian $q(\theta) = \mathcal{N}(\theta_{\text{MAP}}, H^{-1})$ 近似 posterior。Post-hoc 方法, 无需重新训练。

这些方法各有优缺点: MCMC 更精确但计算更贵; deep ensembles 简单但近似粗糙; Laplace 是 post-hoc 但受限于 Gaussian 假设。选择取决于精度需求和计算预算。

5 Question 5: Gradients, Autodiff, and Recursion [20 marks total]

5(a)(i) [3 marks]

Suppose we have a function $f(x, \theta)$ and a probability distribution $p(x|\theta)$, we are interested in computing $\nabla_{\theta} \mathbb{E}_{p(x|\theta)}[f(x, \theta)]$. Assume we can draw samples from $p(x|\theta)$ and assume $f(x, \theta)$ and $p(x|\theta)$ are differentiable with respect to θ .

With no additional assumptions on the form or differentiability of $f(x, \theta)$ or $p(x|\theta)$, define a K -sample Monte Carlo estimator of $\nabla_{\theta} \mathbb{E}_{p(x|\theta)}[f(x, \theta)]$, using samples from $p(x|\theta)$.

使用 **score function estimator** (log-derivative trick / REINFORCE), 推导如下:

$$\begin{aligned}\nabla_{\theta} \mathbb{E}_{p(x|\theta)}[f(x, \theta)] &= \nabla_{\theta} \int p(x|\theta) f(x, \theta) dx \\ &= \int [\nabla_{\theta} p(x|\theta) f(x, \theta) + p(x|\theta) \nabla_{\theta} f(x, \theta)] dx \\ &= \int p(x|\theta) [f(x, \theta) \nabla_{\theta} \log p(x|\theta) + \nabla_{\theta} f(x, \theta)] dx \\ &= \mathbb{E}_{p(x|\theta)}[f(x, \theta) \nabla_{\theta} \log p(x|\theta) + \nabla_{\theta} f(x, \theta)].\end{aligned}$$

其中用了 score function identity $\nabla_{\theta} p(x|\theta) = p(x|\theta) \nabla_{\theta} \log p(x|\theta)$.
 K -sample Monte Carlo estimator:

$$\hat{\nabla} = \frac{1}{K} \sum_{k=1}^K [f(x^{(k)}, \theta) \nabla_{\theta} \log p(x^{(k)}|\theta) + \nabla_{\theta} f(x^{(k)}, \theta)], \quad x^{(k)} \sim p(x|\theta).$$

第一项利用了 score function 把对分布的梯度转化为期望形式; 第二项是 f 对 θ 的直接依赖。

5(a)(ii) [2 marks]

Assume that we can draw samples from $p(x|\theta)$ through a two-step reparameterization procedure, where we instead draw $\epsilon \sim p(\epsilon)$ and obtain values of x as $x = g(\theta, \epsilon)$ for some function g . What additional assumptions would you need to be able to construct a reparameterized Monte Carlo estimator of the gradient using samples from $p(\epsilon)$?

需要以下额外假设:

(1) $f(x, \theta)$ 对 x 可微 (differentiable w.r.t. x): 因为 reparameterisation 后需要通过 $x = g(\theta, \epsilon)$ 反传梯度, 需要 chain rule $\frac{\partial f}{\partial x} \cdot \nabla_{\theta} g$, 要求 f 对其第一个参数可微。

(2) $g(\theta, \epsilon)$ 对 θ 可微 (differentiable reparameterisation function): 变换 $x = g(\theta, \epsilon)$ 必须是 θ 的可微函数, 使得 $\nabla_{\theta} g$ 存在。

这些条件排除了 discrete distributions (discrete x 使 f 对 x 不可微) 和不可微的采样过程。

5(a)(iii) [3 marks]

Under your assumptions in part (ii), define the reparameterized K -sample Monte Carlo estimator for the gradient.

通过 reparameterisation $x = g(\theta, \epsilon)$, 其中 $\epsilon \sim p(\epsilon)$ (与 θ 无关的基分布):

$$\begin{aligned}\nabla_{\theta} \mathbb{E}_{p(x|\theta)}[f(x, \theta)] &= \nabla_{\theta} \mathbb{E}_{p(\epsilon)}[f(g(\theta, \epsilon), \theta)] \\ &= \mathbb{E}_{p(\epsilon)}[\nabla_{\theta} f(g(\theta, \epsilon), \theta)].\end{aligned}$$

K -sample Monte Carlo estimator:

$$\hat{\nabla}_{\text{reparam}} = \frac{1}{K} \sum_{k=1}^K \nabla_{\theta} f(g(\theta, \epsilon^{(k)}), \theta), \quad \epsilon^{(k)} \sim p(\epsilon).$$

其中梯度通过 chain rule 展开为:

$$\nabla_{\theta} f(g(\theta, \epsilon), \theta) = \left. \frac{\partial f}{\partial x} \right|_{x=g(\theta, \epsilon)} \cdot \nabla_{\theta} g(\theta, \epsilon) + \nabla_{\theta} f(x, \theta) \Big|_{x=g(\theta, \epsilon)}.$$

经典例子: $p(x|\theta) = \mathcal{N}(x|\mu, \sigma^2)$, reparameterisation 为 $x = \mu + \sigma\epsilon$, $\epsilon \sim \mathcal{N}(0, 1)$ 。

5(a)(iv) [2 marks]

Assuming both are applicable, when might you prefer one estimator over the other?

Prefer reparameterised estimator (当它适用时): 通常具有显著更低的方差。因为它直接利用了 f 和 g 的梯度信息 (pathwise derivative), 而 score function estimator 的梯度信号来自 $\nabla_{\theta} \log p(x|\theta)$ 与 f 值的乘积, 方差通常很高。在 VAE 训练中, reparameterised estimator (Kingma & Welling, 2014) 是标准选择。

Prefer score function estimator: 当 $f(x, \theta)$ 对 x 不可微 (如 f 包含 discrete / non-differentiable operations), 或者 $p(x|\theta)$ 是 discrete distribution (无法做连续 reparameterisation) 时, score function estimator 是唯一选择。典型场景: reinforcement learning 中 discrete actions 的 policy gradient。Score function estimator 更 general-purpose, 但方差更高, 通常需要 variance reduction (如 baseline)。

5(b)(i) [2 marks]

Now consider the following specific recursive function $f(x)$, where $0 \leq x < 1$:

```
import torch.distributions as dist
def f(x):
    z = dist.Uniform(0, 1).sample()
    if z > x:
        return z
    else:
        return z + f(x)
```

Suppose you wanted to compute $\frac{df}{dx}$. Could you do this with reverse mode autodiff? If not, why not?

不能。原因有二:

(1) **Stochastic branching**: `if z > x` 中的条件分支依赖于随机采样的 z , 每次执行的 computation graph 结构不同 (递归深度不确定)。Reverse mode autodiff 构建的 computation graph 在每次运行后不同, 无法给出一个一致的 df/dx 。

(2) **Sampling** 操作不可微: $z \sim \text{Uniform}(0, 1)$ 的采样本身不是 x 的函数, z 对 x 没有梯度路径。即使固定某次 execution trace, 梯度也无法通过 `dist.Uniform(0,1).sample()` 传播到 x 。
因此 reverse mode autodiff (如 PyTorch autograd) 无法计算 df/dx 。

5(b)(ii) [2 marks]

Could you do this with symbolic differentiation? If not, why not?

不能。原因有二:

(1) 随机递归, 无封闭形式: 函数的递归深度取决于随机采样的 z 值。每次调用 $f(x)$ 可能递归 0 次、1 次、或任意多次, 无法写成固定的确定性符号表达式。

(2) **Stochastic control flow**: `if z > x` 中条件依赖于随机变量 z , symbolic differentiation 无法处理这种 stochastic branching — 它需要一个确定性的表达式来操作。

Symbolic differentiation 要求函数能被表达为一个固定的、确定性的符号表达式, 而 $f(x)$ 是一个 **stochastic program**, 其执行路径在运行时才确定, 不满足 symbolic differentiation 的前提。

5(c) [6 marks]

Still considering the recursive function defined in part (b), let's suppose we want to find values of x for which $f(x) \approx 4$. Using the methods we've studied in this module, how would you propose approaching this problem?

由于传统 gradient-based optimisation 和 symbolic/automatic differentiation 都不可用 (如 5(b) 所述), 需要不依赖梯度的方法。结合本课程内容, 有以下方案:

方法一: **Finite differences + stochastic optimisation**

定义损失 $L(x) = \mathbb{E}[(f(x) - 4)^2]$ 。

Step 1: Monte Carlo 估计 $L(x)$: 对固定 x , 运行 $f(x)$ 共 S 次, 取 $\hat{L}(x) = \frac{1}{S} \sum_{s=1}^S (f_s(x) - 4)^2$ 。

Step 2: Finite differences 近似梯度:

$$\widehat{\nabla_x L}(x) \approx \frac{\hat{L}(x + \delta) - \hat{L}(x - \delta)}{2\delta}.$$

Step 3: Stochastic gradient descent 更新: $x \leftarrow x - \eta \widehat{\nabla_x L}(x)$ 。

方法二: **Bayesian optimisation**

将 $L(x)$ 视为 expensive noisy black-box function, $x \in [0, 1)$ (一维):

1. 在若干 x 值处评估 $\hat{L}(x)$;
2. 拟合 GP surrogate model;
3. 用 acquisition function (如 Expected Improvement) 选择下一个 x ;
4. 重复直到 $L(x) \approx 0$ 。

一维搜索空间使得 Bayesian optimisation 非常高效。

补充: 解析验证

可以推导 $m(x) = \mathbb{E}[f(x)]$ 的解析形式来验证数值结果。当 $z > x$ (概率 $1 - x$) 返回 z ; 当

$z \leq x$ (概率 x) 返回 $z + f(x)$ (递归)。设 $m(x) = \mathbb{E}[f(x)]$:

$$\begin{aligned} m(x) &= (1-x) \mathbb{E}[z|z > x] + x (\mathbb{E}[z|z \leq x] + m(x)) \\ &= (1-x) \cdot \frac{1+x}{2} + x \cdot \frac{x}{2} + x \cdot m(x) \\ &= \frac{(1-x)(1+x)}{2} + \frac{x^2}{2} + x \cdot m(x) \\ &= \frac{1-x^2+x^2}{2} + x \cdot m(x) = \frac{1}{2} + x \cdot m(x). \end{aligned}$$

解得 $m(x)(1-x) = \frac{1}{2}$, 即 $m(x) = \frac{1}{2(1-x)}$ 。令 $m(x) = 4$, 解得 $x = \frac{7}{8} = 0.875$ 。

这个解析解可以作为数值方法的 ground truth 验证。